Machine Learning

Machine learning with scikit-learn — Lecture 19 (Chapter 9, Part 2)

Amir Farbin

Part 2 — Machine learning with scikit-learn

Why a framework

- We built Fisher's discriminant **by hand**: none of it is difficult
- A framework makes it **fast**, and lets you **swap algorithms** with the same few lines, so you can try several and **compare** them
- **scikit-learn** is the standard for this kind of machine learning
- Its documentation is organised by **task**: **classification** (predict a category), **regression** (predict a number), **clustering** (group examples without labels), dimensionality reduction, model selection, metrics, preprocessing, pipelines

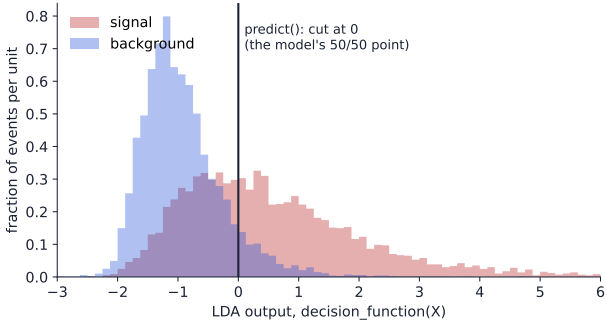
One interface, many algorithms

- Every model, an **estimator** in scikit-learn's words, is a **class**: make an **instance** (one object of that class), then
  - `fit(X, y)`: **train**: learn the **parameters** (the numbers the model adjusts), which stay in that instance
  - `decision_function(X)`: the continuous output, a **score** (a test statistic)
  - `predict(X)`: the class, from cutting the score at a default **threshold**
- A **pipeline** chains steps (scale, reduce, classify) without knowing which model it holds: **polymorphism**, as in Chapter 4's gradebook

Formulating the problem

- **X:** the 18 **features** (the inputs). **y:** the `signal` column, the **label** or **target** (the answer)
- **Never give the answer as an input.** A score of 100% means: look for the answer, or something perfectly correlated with it, inside X
- **Training set:** the events the model learns from (4 million). **Test set:** events held back to judge it (1 million). Splitting is just slicing (`train_test_split` adds shuffling)
- Rates are per class: **TPR** (true positive rate, the fraction of signal kept) on signal only, **FPR** (false positive rate, the fraction of background kept) on background only

LDA is Lecture 17's Fisher discriminant

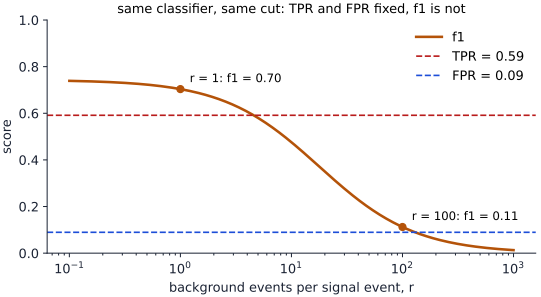


- **LDA** (linear discriminant analysis): scikit-learn's Fisher discriminant. Same direction as our hand-built w, same AUC. Its zero is the model's **50/50 point**
- `predict` = `decision_function > 0`. A default, **not** the best cut

ROC and AUC from the library

- **ROC curve** (receiver operating characteristic): TPR against FPR as the threshold moves. **AUC:** the area under it, 1 = perfect, 0.5 = guessing
- `roc_curve(y, score)`, `auc(fpr, tpr)`: what you wrote in Lecture 17
- Give it the **score**, not `predict`: 0/1 predictions make a two-segment "curve"
- The thresholds it returns turn "I want this TPR" into a cut
- AUC on train vs test: **0.837 vs 0.838**, so no **overtraining** (doing better on the training set than on new data)

TPR and FPR don't depend on the mixture. f1 does



- **Precision:** of what you kept, the fraction that is signal. **Recall** $=$ TPR. **f1:** combines them, $2 \cdot \text{precision} \cdot \text{recall}/(\text{precision} + \text{recall})$. **Accuracy:** fraction of all events classified right
- With r background per signal: $f_1 = 2\,\text{TPR}/\left(2\,\text{TPR} + (1 - \text{TPR}) + r\,\text{FPR}\right)$

Tuning needs its own sample

- Picking the threshold that maximises f1 **on the test sample** is fitting to the test sample
- Tune on a **validation set** (a third sample, for choices), report once on the **test set**
- The same goes for choosing between algorithms, and their settings, the **hyperparameters** (chosen by you, not learned in `fit`)

scikit-learn's own examples

- Standard datasets for learning and benchmarking: **iris** (150 flowers, 4 measurements) and **digits** (1,797 images, $8 \times 8 = 64$ numbers)
- `SVC` (a **support vector classifier**, the SVM from Lecture 17) on digits: the same pattern, instance, `fit`, `predict`
- One correct prediction is not a test: on 297 **held-out** digits (never used in training) it gets **95%** accuracy
- **Lab 8:** three algorithms on SUSY, compare ROC curves, carry the best to a significance

Vocabulary

- **estimator:** a scikit-learn model. **class, instance, fit (train), predict, score, threshold**
- **parameters:** learned in `fit`. **hyperparameters:** chosen by you
- **features (X):** the inputs. **label / target (y):** the answer
- **classification, regression, clustering**
- **training set, validation set, test set, held-out data, overtraining**
- **TPR (recall), FPR, ROC curve, AUC, precision, f1, accuracy**
- **LDA:** linear discriminant analysis (Fisher). **SVC:** support vector classifier
- **pipeline:** steps chained into one estimator. **polymorphism:** one interface, many classes